EventSphere – Screens, Roles and Flow (Draft for Review)

Based on the UI references shared so far. All open questions are now decided (section 7). Review it, then we start Phase 1.

1. Roles and Login

Three roles share one login page:

Role	Lands on	Purpose
User (attendee)	Home page	Browse events, book tickets, view My Tickets
Organizer	Organizer Dashboard	Create events, see attendees, monitor check-ins
Gate Staff	Scanner page	Scan QR codes at the venue entrance

Role selector: three role cards inside the login card (Organizer, User, Gate Scanner), as in the login design in section 8. After login, each role sees only its own screens and routes.

2. Organizer Side

Layout follows the organizer dashboard reference: left sidebar, white cards, plum accents.

Sidebar items (Analytics and Settings removed):

- Dashboard
- My Events
- Attendees
- Check-ins

Screens

Screen	What it shows
Dashboard	Welcome message; four cards (Total Events, Total Attendees, Upcoming Events, Check-ins Today); Recent Events list with a View button

Screen	What it shows
My Events	List of the organizer's events with Create Event button and Edit option
Create / Edit Event	Title, description, event type, date and time, venue, ticket types (name, price, quantity) with Add Ticket Type, Create Event button
Attendees	Table with name, ticket type, status (Checked In / Pending), check-in time; search by name, email or ticket ID; filter
Check-ins	Live check-in status and a capacity view (available / booked / checked in). Scanning itself happens on the Gate Scanner page

3. User (Attendee) Side

Layout follows the second reference image: left sidebar, hero banner with search, event cards.

Sidebar items (Organizer Dashboard and Gate Scanner removed):

- Home
- Events
- My Tickets
- Favorites
- Profile

Screens

Screen	What it shows
Home	Search bar; hero banner (Discover Amazing Events Near You); Featured Events cards with Book Now; Popular Events with category chips (All, Music, Conferences, Workshops, Festivals); a short "Why choose EventSphere" strip
All Events	Reached via the Events sidebar item or View All. List of events with category filters and price range

Screen	What it shows
Event Details	Banner image, title, category tag, date, venue, About section, ticket type selection with remaining count (e.g. General, VIP), Proceed to Checkout
Checkout	Order summary, ticket type and quantity, subtotal, platform fee, total, a hold countdown timer, and Pay button that opens the Razorpay popup
Booking Confirmed	Success message, event summary, QR code, booking ID, Download Ticket and View in My Tickets
My Tickets	Tabs for Upcoming and Past; each ticket card has a QR and View Ticket
Favorites	Events the user has hearted
Profile	Basic account details

4. Gate Staff Side

A single focused page:

- Scan QR (camera) and a manual ticket ID field with Check In button
- Clear result screen: **green** Valid (name, ticket type), **red** Already Used (and logged as a duplicate attempt), **red** Invalid
- Small list of recent scans

5. End-to-End Flow

1. Organizer logs in, creates an event with ticket types and quantities.
2. User logs in, browses Home or All Events, opens an event.
3. User picks a ticket type and quantity and proceeds to checkout. The system holds that quantity for 5 minutes (Redis) and shows a countdown.
4. User pays through Razorpay. The booking and tickets are saved together in one MongoDB transaction.
5. User sees Booking Confirmed with a signed QR, and the ticket appears in My Tickets.

6. At the venue, gate staff scans the QR. Valid tickets are marked used; repeat scans are rejected and logged.
7. Organizer sees updated attendee statuses, check-in counts and dashboard numbers.

6. What Is In and Out (5-Hour Build)

In: three-role login, organizer dashboard / my events / create event / attendees / check-ins, user home / all events / event details / checkout / confirmation / my tickets, gate scanner, Redis hold, Razorpay test payment, signed QR, duplicate scan rejection.

Out for now: Analytics, Settings, Gate Scanner and Organizer Dashboard links for users, full interactive seat picking, AWS ECS deployment (Dockerfiles and GitHub Actions stay in the repo).

Optional if time remains: Favorites, rotating QR (30 seconds), live updates, k6 load test.

7. Final Decisions

1. **Color theme:** plum, cream and sunset gradient from the login design, used on every screen.
2. **Tickets:** users choose ticket type and quantity (no seat picking). The organizer sees a live capacity view.
3. **Check-ins page (organizer):** shows status only. All scanning happens on the Gate Scanner page.
4. **Gate staff:** one demo account that can scan any event.
5. **Accounts:** one pre-seeded demo account per role, plus a Sign up link for users.
6. **Favorites and Profile:** Favorites is a heart toggle; Profile is a basic details page.
7. **Event images:** the organizer pastes an image URL; default images are the fallback.
8. **Ticket download:** QR on screen plus a Download Ticket button that saves an image.
9. **Kept for looks:** the "Login with different account" link stays on the login page as a visual element only (not functional).

8. Login Page and Design System

Login page (split layout, from the login design):

- Left side: concert photo with sunset gradient, EventSphere logo, headline "Great Events Start Here", three feature points (Fake-proof QR Tickets, Real-time Seat Holds, Fast Check-in).
- Right side: "Welcome to EventSphere" card with three role cards (Organizer, User, Gate Scanner), a Quick Login box with email and password, a plum Login button, and a "Secure & Encrypted" note.
- The "or / Login with different account" divider and link stay for looks only; they do nothing.

Palette (approximate, exact values to be taken from the login image):

Color	Used for	Approx. value
Deep plum	Buttons, headings, logo, active sidebar item	#5A2A55
Warm cream	Page and card backgrounds	#FBF8F4
Lavender / pink tint	Selected role card, chips, highlights	light tints of plum
Sunset gradient	Login hero, Home banner	peach, pink, purple

Layout rules for every logged-in page:

- Top right corner: profile area with name, role and Logout.
- Bottom left of the sidebar: the small EventSphere badge ("Secure · Scalable · Built for Real Events").
- Sidebars follow sections 2 and 3.

9. Demo Access for Recruiters

Recruiters will not know any logins, so access must take one click:

- Clicking a role card pre-fills that role's demo email and password; the visitor only presses Login.
- A short line under the form says "Demo mode: credentials are pre-filled for each role."
- The database is seeded with sample events, bookings and attendees so no page looks empty.
- Razorpay stays in test mode; the test card details appear in the README and as a hint on checkout.
- Users can also create their own account through Sign up.

10. Hosting and Delivery

- Live link: frontend on Vercel, backend on Render or Railway, MongoDB Atlas, free Redis (Upstash).
- Repo includes Dockerfiles, docker-compose and a GitHub Actions workflow to cover the required stack.
- README includes screenshots or a short demo video, setup steps, architecture summary, demo credentials and the Razorpay test card.
- Optional if time remains: a short k6 load-test result showing no double-booking.

11. Build Order (5 Hours)

Time	What	Done when
0:00 - 0:30	Setup: monorepo, Docker (Mongo replica set + Redis), skeletons	App runs, /health works
0:30 - 1:15	Auth, three roles, login page, sidebars, seed data	Each role logs in and sees its own layout
1:15 - 2:00	Events: organizer create/edit, user Home, All Events, Event Details	Organizer creates an event, user sees it
2:00 - 2:45	Redis hold with countdown on checkout	Same tickets cannot be held twice; hold expires
2:45 - 3:30	Razorpay test payment and transactional booking	One payment creates one booking and its tickets
3:30 - 4:15	Signed QR, My Tickets, Gate Scanner with duplicate rejection	Second scan shows Already Used
4:15 - 4:40	Organizer dashboard, attendees, check-ins page	Numbers and statuses match real data
4:40 - 5:00	Deploy, README, screenshots	Live link works

12. Tech Stack

Area	Tools
Frontend	React (Vite), Tailwind CSS, React Router, QR generator and QR scanner libraries
Backend	Node.js, Express, JWT (login and signed QR tickets), bcrypt
Database	MongoDB with Mongoose (transactions)
Holds	Redis (5-minute ticket holds)
Payments	Razorpay (test mode)
DevOps	Docker, Docker Compose, GitHub Actions
Testing	k6 load test (optional)
Hosting	Vercel, Render or Railway, MongoDB Atlas, Upstash Redis
Build tool	Antigravity

Not in this version: AWS ECS deployment and WebSockets (listed as future improvements).